arguments for all the fields: `first_name`, `last_name`, and `birth`. If you miss one, you will only receive an error when Django executes the underlying database statement at run time.

Field types map to corresponding Python values. The names in `Musician` are `CharField` attributes and map to Python strings, while `birth` is a `DateField` that maps to a Python `date` object.

To create your first Musician, run the Django REPL, using the `shell` management command; import the `Musician` object from `bands.models`; and call the `.create()` method, with arguments for your favorite musician:

```
(venv) RiffMates$ python manage.py shell
Python 3.12.1
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from bands.models import Musician
>>> from datetime import date
>>> steve = Musician.objects.create(first_name="Steve",
➥ last_name="Vai", birth=date(1960, 6, 6))
```

Using the .objects query manager to create a musician passing in string and date keyword argument values

The `.create()` call inserts the content into the database and returns a corresponding `Musician` object. You can display the value of `steve` in the REPL:

```
>>> steve
<Musician: Musician object (1)>
```

The result isn't too friendly. When you display an object like this, the Django REPL casts the object into a string to show the result. The `Model` base class implements `.__str__()`, the special method called when an object gets cast to a string. The provided implementation shows the class name and the object's `id` in parentheses. Your first `Musician` object got an `id` of `1`.

To make it easier to view and debug your objects, it is best practice to override the base class's `.__str__()` method. Exit the Django shell, open RiffMates/bands/models.py, and add the contents of listing 4.3 to your `Musician` class, as shown in the following listing.

Listing 4.3 Overriding `.__str__()` in your `Musician` class

```
# RiffMates/bands/models.py
...
class Musican(models.Model):
    ...
    def __str__(self):
        return f"Musician(id={self.id}, last_name={self.last_name})"
```

Override the special method called when casting Musician to string.

Save the file and re-enter the Django shell. The database is meant as permanent storage; you've already created Mr. Vai, and now you want to fetch him back. The `.objects` query manager provides several methods for querying data out of the